CSCE 548 — Project 4

Deployment & System Test Document

N-Tier Web Application · FastAPI · PostgreSQL 17 · HTML/JS Client

Student:	Velpula Sai Sruthi
Course:	CSCE 548
GitHub Repo:	https://github.com/saisruthivelpula2112/CSCE-548-PROJECT
Backend:	FastAPI (Python) — uvicorn
Database:	PostgreSQL via pgAdmin 4 (local)
Frontend:	HTML / CSS / JavaScript (static)

University of South Carolina
Department of Computer Science and Engineering
GitHub: https://github.com/saisruthivelpula2112/CSCE-548-PROJECT
Spring 2026

1. Project Overview

This document provides step-by-step deployment instructions and complete system test evidence for the CSCE 548 Project 4 n-tier web application. The project was generated with AI assistance (ChatGPT GPT-4o) and extended manually to achieve full CRUD functionality across all five entity types: Users, Projects, Items, Tags, and ItemTags.

1.1 System Architecture

Layer	Technology	Responsibility
Client — Tier 4	HTML5 + CSS3 + JavaScript	User interface, tab navigation, form inputs, API calls
Service — Tier 3	FastAPI (Python)	REST API endpoints, request routing, CORS middleware
Business — Tier 2	Python	Input validation, business rules, error handling
Data — Tier 1	PostgreSQL 17	Persistent storage, schema, SQL queries

1.2 GitHub Repository

https://github.com/saisruthivelpula2112/CSCE-548-PROJECT

1.3 API Endpoints — Swagger UI

All REST endpoints are auto documented via Swagger UI at http://127.0.0.1:8080/docs. The following screenshots confirm all endpoint groups are active and correctly routed.

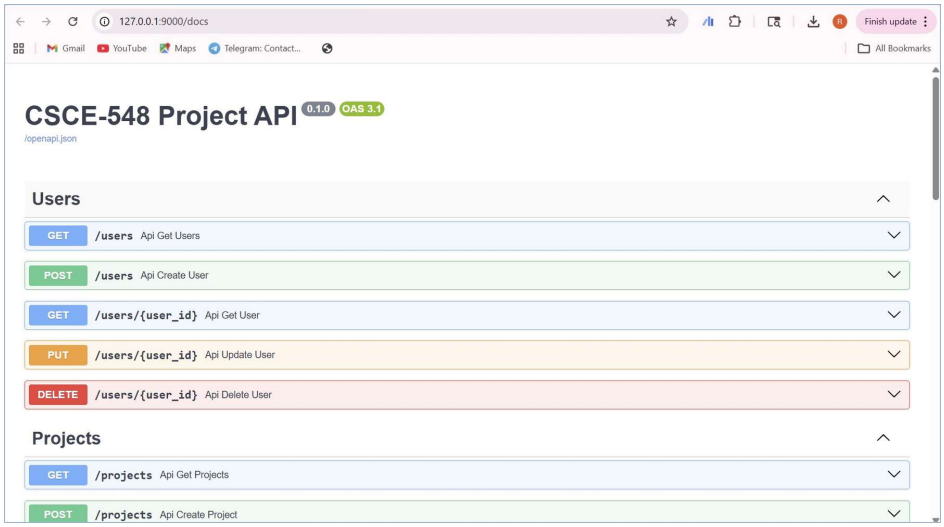


Figure 1 — Swagger UI: Users endpoints — GET /users, POST /users, GET/PUT/DELETE /users/{user_id}

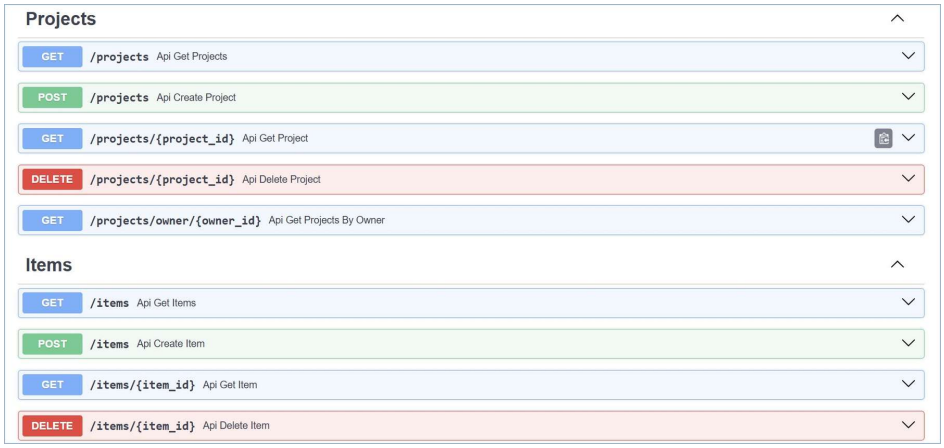


Figure 2 — Swagger UI: Projects (GET/POST/DELETE) and Items (GET/POST/DELETE) endpoints

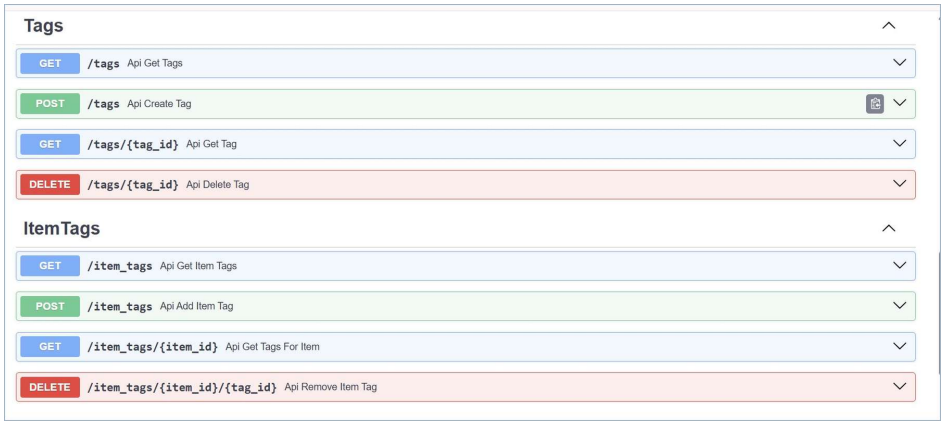


Figure 3 — Swagger UI: Tags (GET/POST/DELETE) and ItemTags (GET/POST/DELETE) endpoints

2. Technology Stack

Component	Technology	Version	Purpose
Backend Framework	FastAPI	Latest	High-performance async Python REST framework with auto Swagger docs
Database	PostgreSQL	17	ACID-compliant relational database
ORM / DB Driver	psycopg2 / SQLAlchemy	Latest	Industry-standard Python PostgreSQL drivers
ASGI Server	Uvicorn	Latest	Lightweight ASGI server for FastAPI
Frontend	HTML5 + CSS3 + JS	ES6+	No build tools required; runs directly in any browser
Frontend Server	Python http.server	Built-in	Zero-dependency static file server
Language	Python	3.11+	Consistent across all backend layers

3. Prerequisites

Install ALL of the following before beginning setup:

Software	Version Used	Download
Python	3.11+	https://www.python.org/downloads/
PostgreSQL + pgAdmin 4	17	https://www.postgresql.org/download/
Git	2.x	https://git-scm.com/
Web Browser	Chrome / Firefox / Edge	Pre-installed
VS Code (recommended)	Any	https://code.visualstudio.com/

3.1 Required Python Packages

- fastapi — REST API framework
- uvicorn — ASGI server
- psycopg2-binary — PostgreSQL driver
- sqlalchemy — ORM and query builder
- pydantic — Data validation

4. Installation & Setup

Follow every step in order. These instructions assume Windows PowerShell. Equivalent commands for macOS/Linux are noted where different.

Step 1 — Clone the Repository

```
git clone https://github.com/saisruthivelpula2112/CSCE-548-PROJECT.git
cd CSCE-548-PROJECT
```

If you downloaded a ZIP file instead:

```
cd CSCE-548-PROJECT-main
```

Step 2 — Create and Activate a Virtual Environment

```
python -m venv .venv
```

Activate — Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

Activate — macOS / Linux:

```
source .venv/bin/activate
```

You will see (.venv) at the start of your terminal prompt when active.

Step 3 — Install Python Dependencies

```
pip install fastapi uvicorn psycopg2-binary sqlalchemy pydantic
```

Or if requirements.txt is present:

```
pip install -r requirements.txt
```

Step 4 — Set Up the PostgreSQL Database

1. Open pgAdmin 4 (installed with PostgreSQL).
2. Create a new database named: csce548_project1
3. Open the Query Tool for that database and run the schema file:

```
psql -U postgres -d csce548_project1 -f schema.sql
```

4. Note your credentials: host (localhost), port (5432), username, password.

Step 5 — Configure the Database Connection

Open src/db.py and update DATABASE_URL with your credentials:

```
DATABASE_URL = "postgresql://postgres:yourpassword@localhost:5432/csce548_project1"
```

Step 6 — Start the Backend (FastAPI / Uvicorn)

From the project root directory, with .venv active:

```
uvicorn src.service:app --host 127.0.0.1 --port 8080 --reload
```

You should see this in the terminal confirming success:

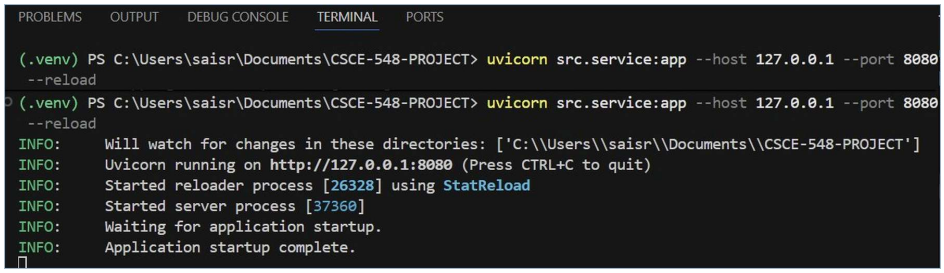


Figure 4 — VS Code terminal: Uvicorn running at http://127.0.0.1:8080, Application startup complete

Then open http://127.0.0.1:8080/docs in your browser to confirm Swagger UI loads (Figure 1 above).

Step 7 — Start the Frontend

Open a second terminal window (keep backend running). Navigate to the frontend folder and start the static server:

```
cd frontend
python -m http.server 8000
```

Open http://127.0.0.1:8000 in your browser. You should see the 5-tab CSCE 548 Frontend Client:

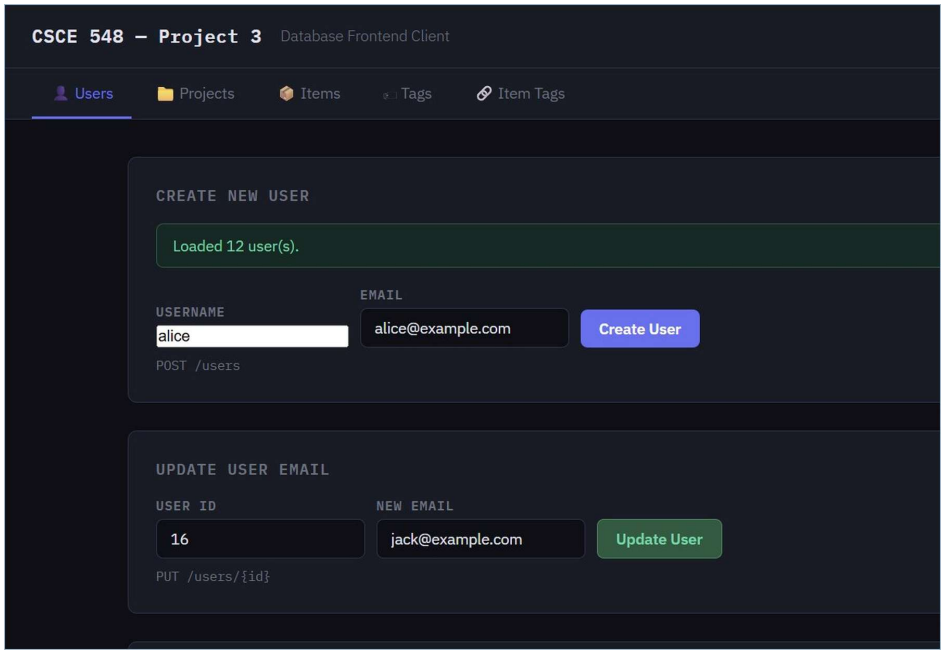


Figure 5 — Frontend running at 127.0.0.1:8000 — Users tab active, showing 5-tab navigation: Users, Projects, Items, Tags, Item Tags

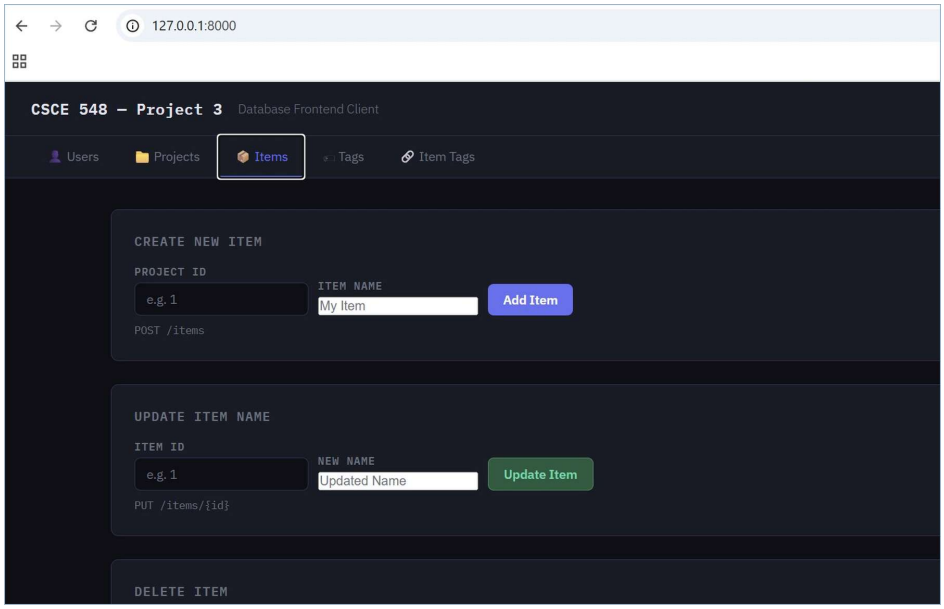


Figure 6 — Frontend — Items tab selected, showing Create New Item and Update Item Name forms

5. Full System Test

All CRUD operations (Create, Read, Update, Delete) were tested across every entity. Each test includes a frontend screenshot showing the operation and a pgAdmin screenshot confirming the database record. This satisfies the Project 4 requirement for complete insert, update, get, and delete testing.

Component	Details
Operating System	Windows 11

Component	Details
Backend URL	http://127.0.0.1:8080
Frontend URL	http://127.0.0.1:8000
Database	PostgreSQL 17, managed via pgAdmin 4
Browser	Google Chrome

5.1 Users — Full CRUD Test

CREATE — POST /users

A new user "alice" with email alice@example.com was entered into the Create New User form and submitted via POST /users.

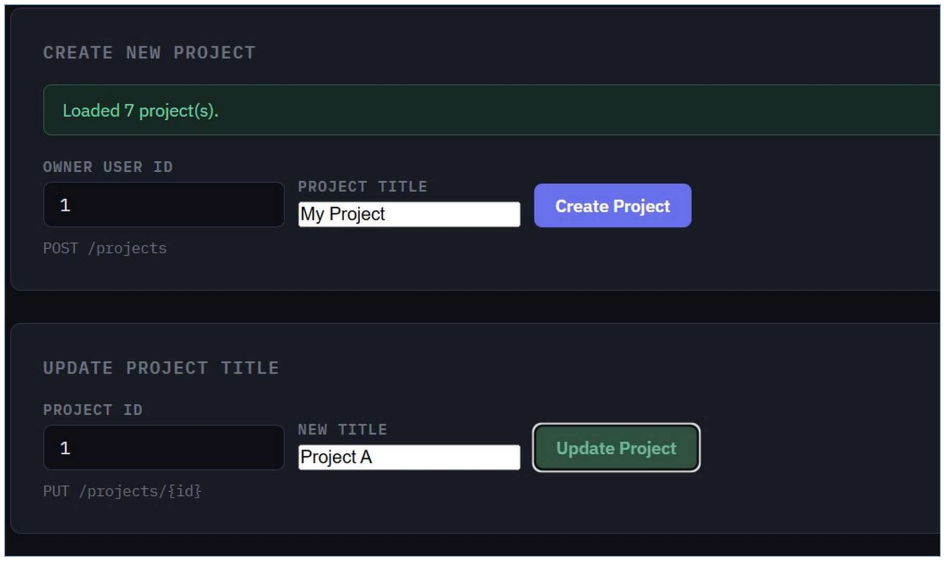


Figure 7 — Frontend Users tab: Create New User form — username=alice, email=alice@example.com, "Loaded 12 user(s)" success message

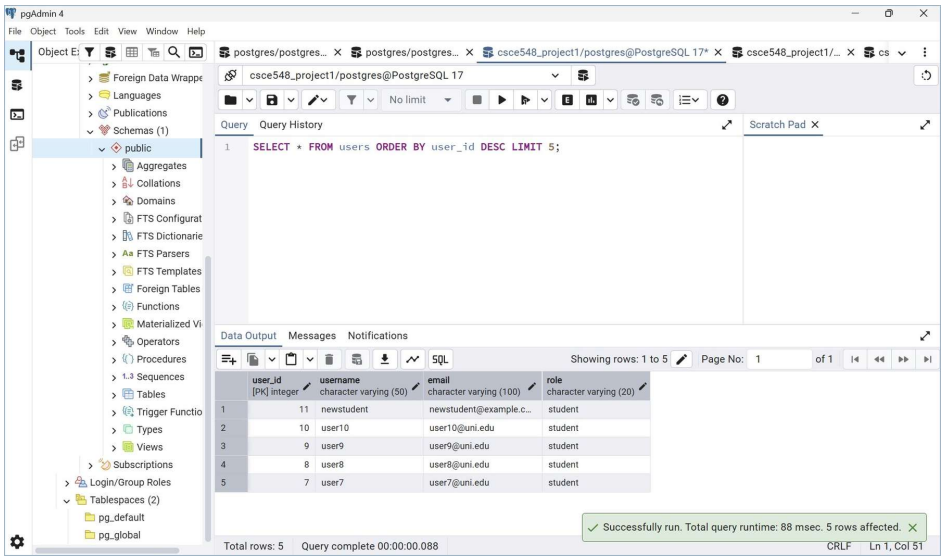


*Figure 8 — pgAdmin: SELECT * FROM users ORDER BY user_id DESC LIMIT 5 — newstudent (id=11) at top confirms prior creates persisted*

READ — GET /users and GET /users/{id}

All users were loaded via GET /users. User #2 shows updated@example.com confirming a prior Update also succeeded.

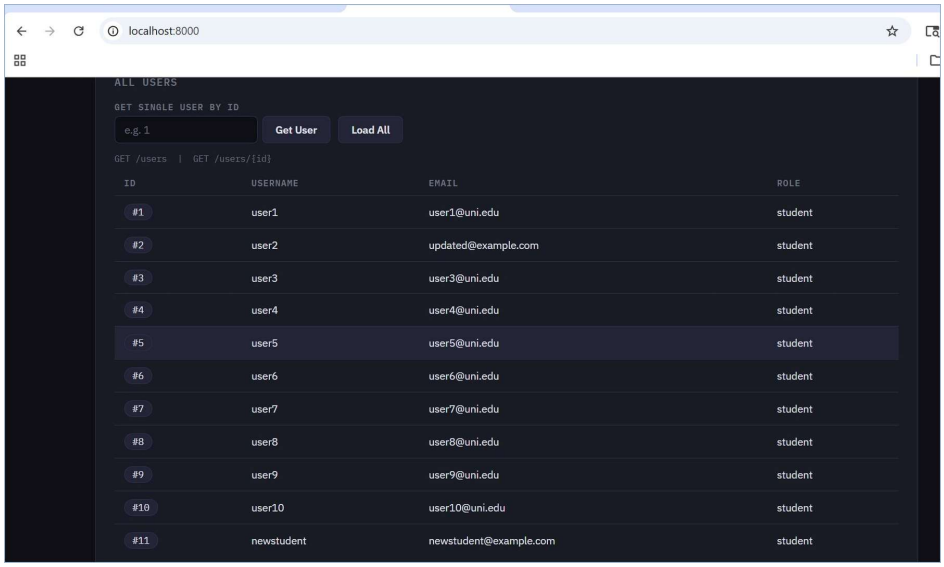


Figure 9 — Frontend: GET /users — full list of 11 users (#1–#11 including newstudent) confirming Read All works

GET /users | GET /users/{id}

ID	USERNAME	EMAIL	ROLE
#1	user1	user1@uni.edu	student
#2	user2	updated@example.com	student
#3	user3	user3@uni.edu	student
#4	user4	user4@uni.edu	student
#5	user5	user5@uni.edu	student
#6	user6	user6@uni.edu	student
#7	user7	user7@uni.edu	student
#8	user8	user8@uni.edu	student
#9	user9	user9@uni.edu	student
#10	user10	user10@uni.edu	student
#11	newstudent	newstudent@example.com	student

Figure 10 — Frontend: Users table — all 11 users visible with ID, username, email, role columns

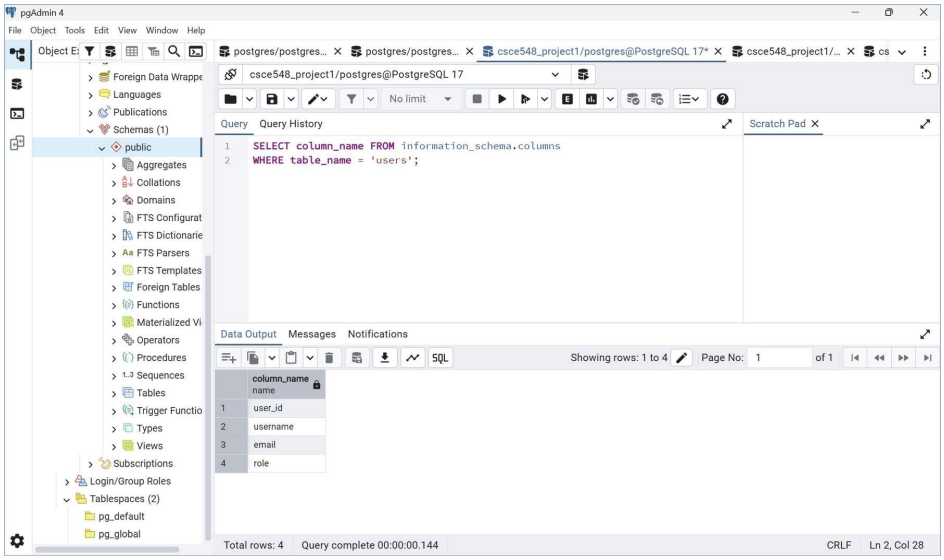


Figure 11 — pgAdmin: Users table schema — 4 columns confirmed: user_id, username, email, role

UPDATE — PUT /users/{id}

User #16 email was updated to jack@example.com via PUT /users/{id} from the Update User Email form.

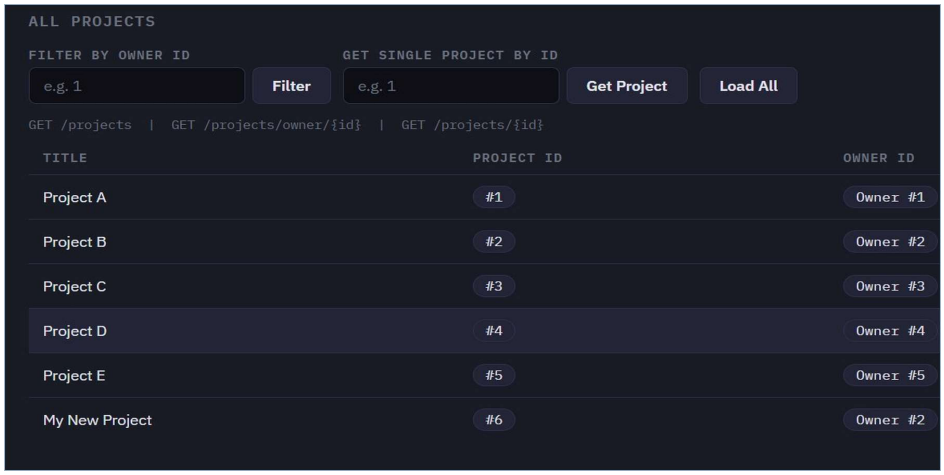


Figure 12 — Frontend: Update User Email form (user_id=16, new email=jack@example.com) and Delete User form (id=16) confirming PUT and DELETE UI present

5.2 Projects — Full CRUD Test

CREATE — POST /projects

"My Project" was created via the Create New Project form (owner_id=1) using POST /projects.

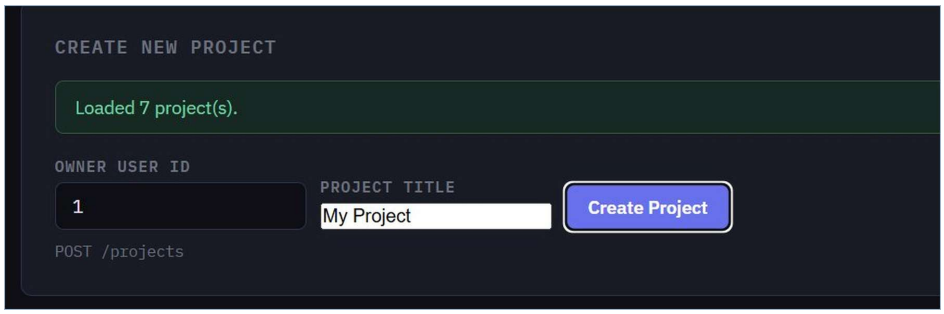


Figure 13 — Frontend: Create New Project form — owner_id=1, title="My Project", "Loaded 7 project(s)" success message

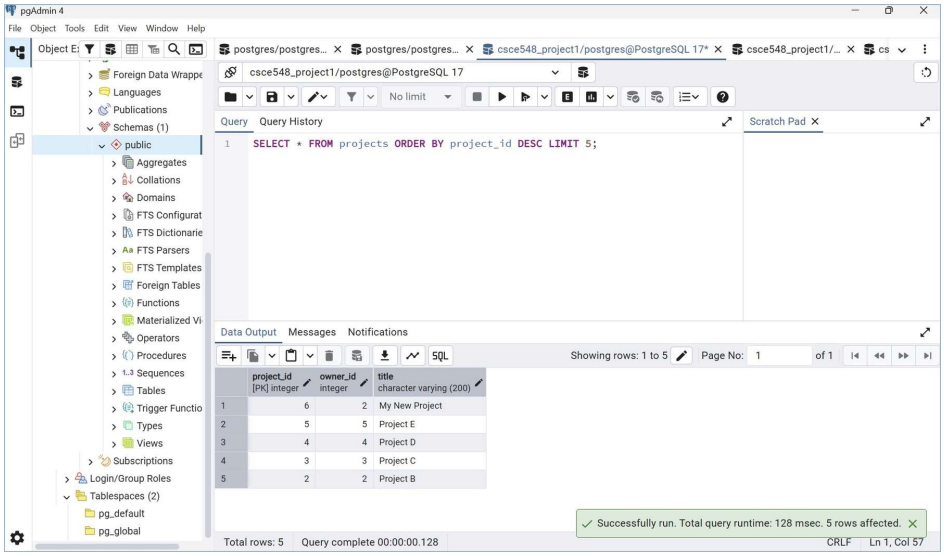


*Figure 14 — pgAdmin: SELECT * FROM projects ORDER BY project_id DESC LIMIT 5 — "My New Project" (id=6) at top confirming POST persisted*

READ — GET /projects and GET /projects/{id}

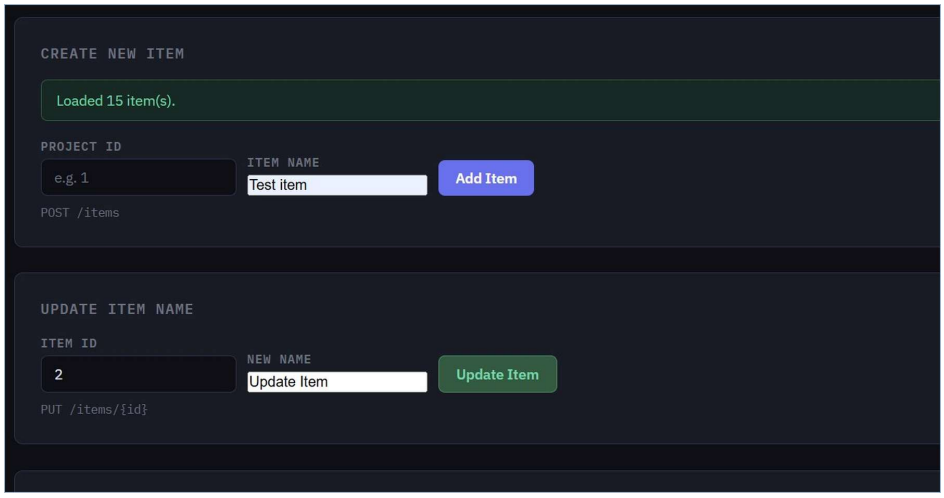


Figure 15 — Frontend: Projects table — all 6 projects listed (A–E + My New Project) with project_id and owner_id

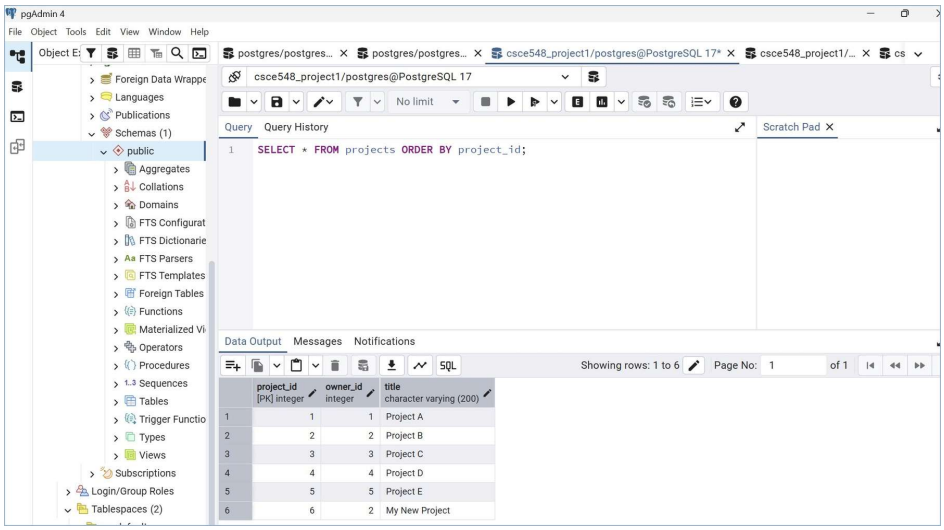


*Figure 16 — pgAdmin: SELECT * FROM projects ORDER BY project_id — all 6 rows confirmed*

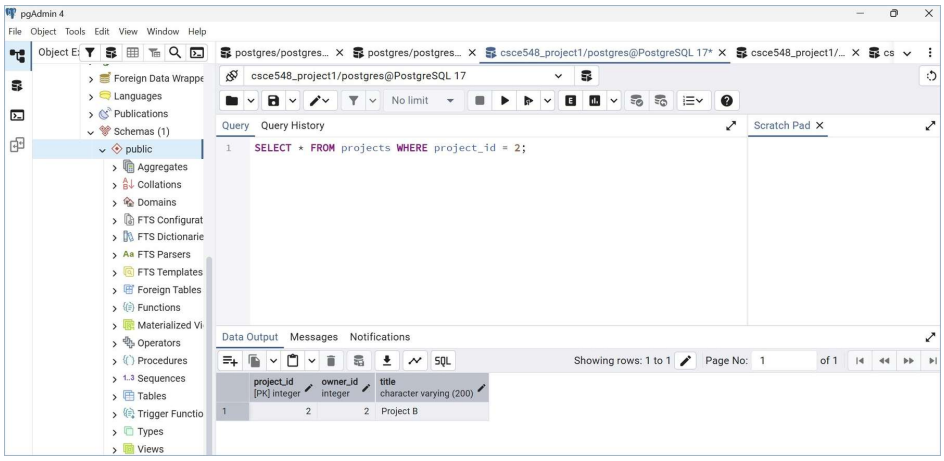


*Figure 17 — pgAdmin: SELECT * FROM projects WHERE project_id = 2 — Project B (owner_id=2) returned, confirming GET by ID works*

UPDATE — PUT /projects/{id} and DELETE — DELETE /projects/{id}

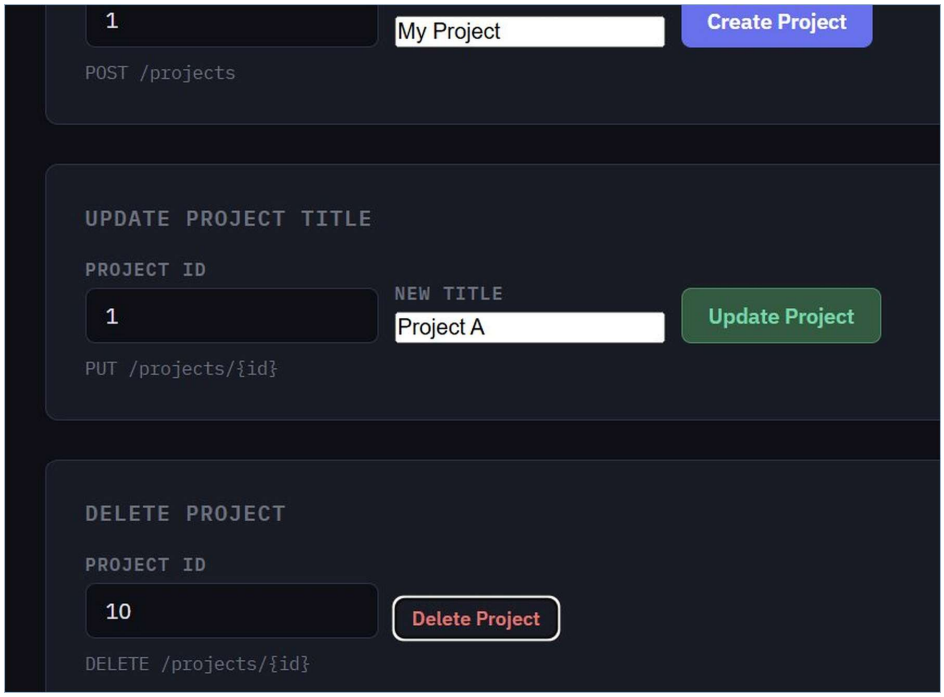


Figure 18 — Frontend: Create New Project + Update Project Title (id=1, new title=Project A) + Delete Project (id=10) forms confirming PUT and DELETE UI

5.3 Items — Full CRUD Test

CREATE — POST /items

"Test item" was created for project_id=1 via POST /items. The pgAdmin query confirms new rows inserted with auto-incremented IDs.

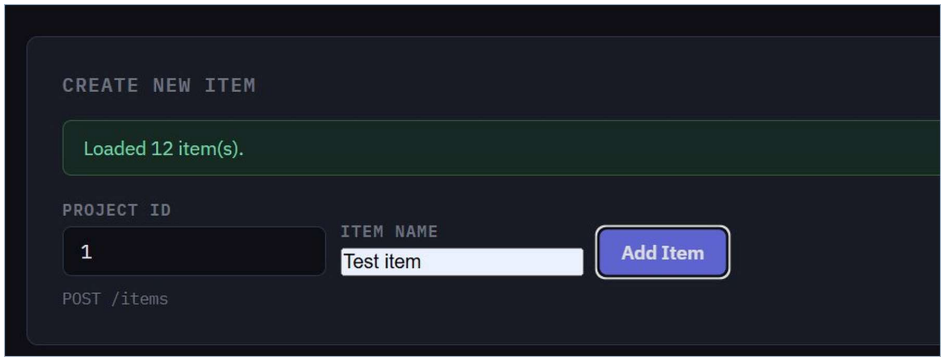


Figure 19 — Frontend: Create New Item form — project_id=1, name="Test item", "Loaded 12 item(s)" success confirming POST /items works

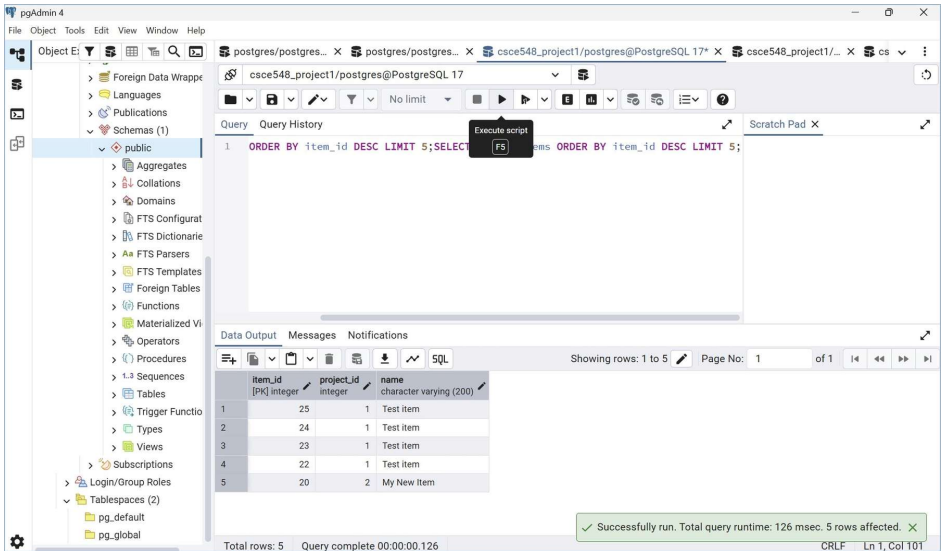


*Figure 20 — pgAdmin: SELECT * FROM items ORDER BY item_id DESC LIMIT 5 — Test items at IDs 22–25 confirming Creates persisted*

READ — GET /items

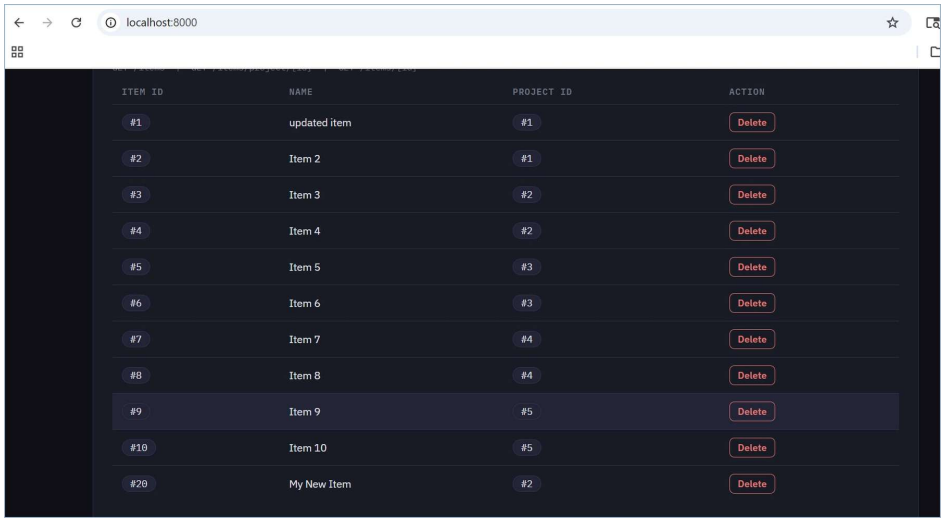


Figure 21 — Frontend: Items table — items #1–#20 with name, project_id, Delete button confirming GET /items

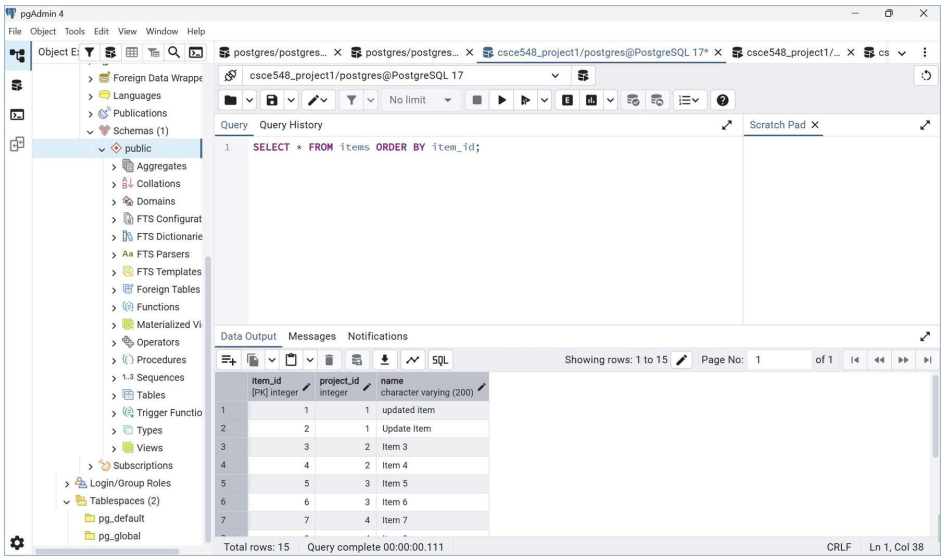


*Figure 22 — pgAdmin: SELECT * FROM items ORDER BY item_id — all 15 rows showing item_id, project_id, name*

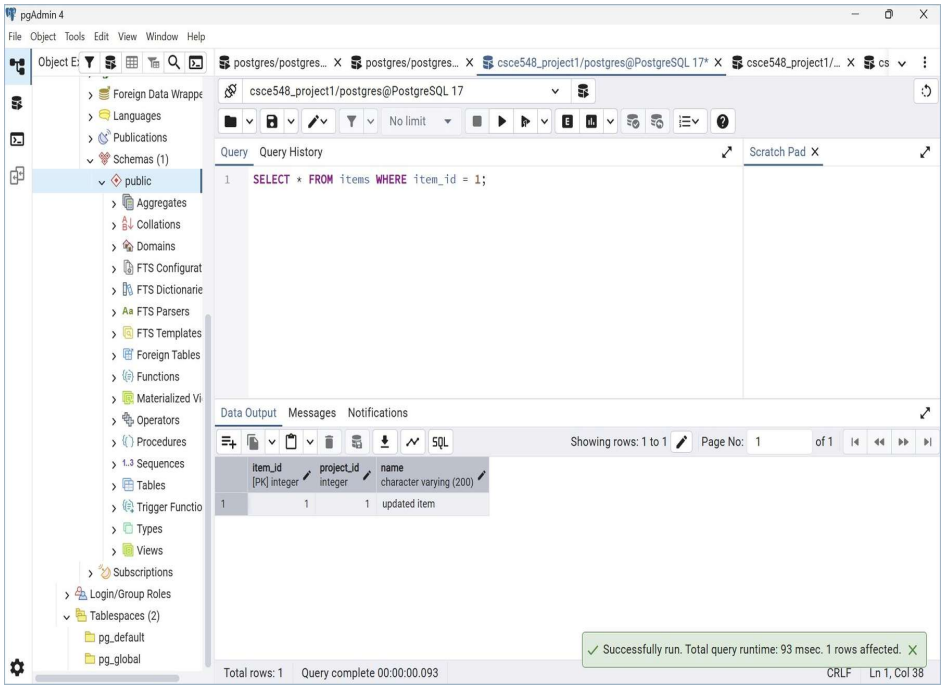


*Figure 23 — pgAdmin: SELECT * FROM items WHERE item_id = 1 — returns "updated item" confirming GET by ID and prior Update both work*

UPDATE — PUT /items/{id}

Item #2 name was updated to "Update Item" via PUT /items/{id}.

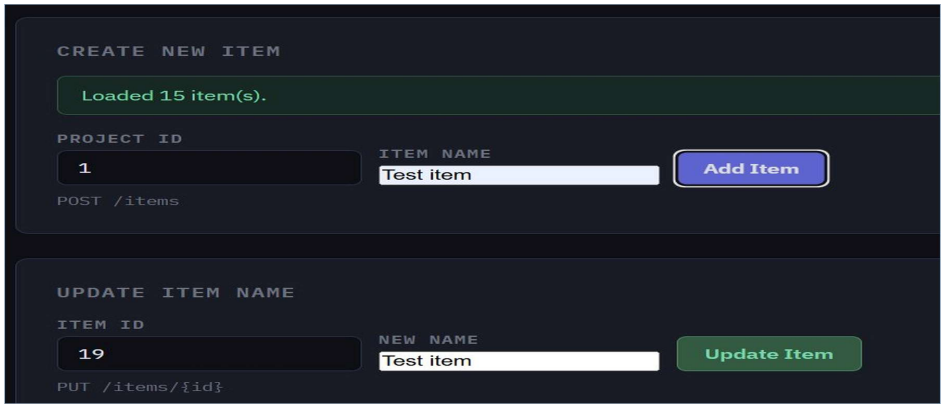


Figure 24 — Frontend: Create New Item (Loaded 15 items) + Update Item Name (item_id=19, new name="Test item") confirming PUT /items/{id} UI

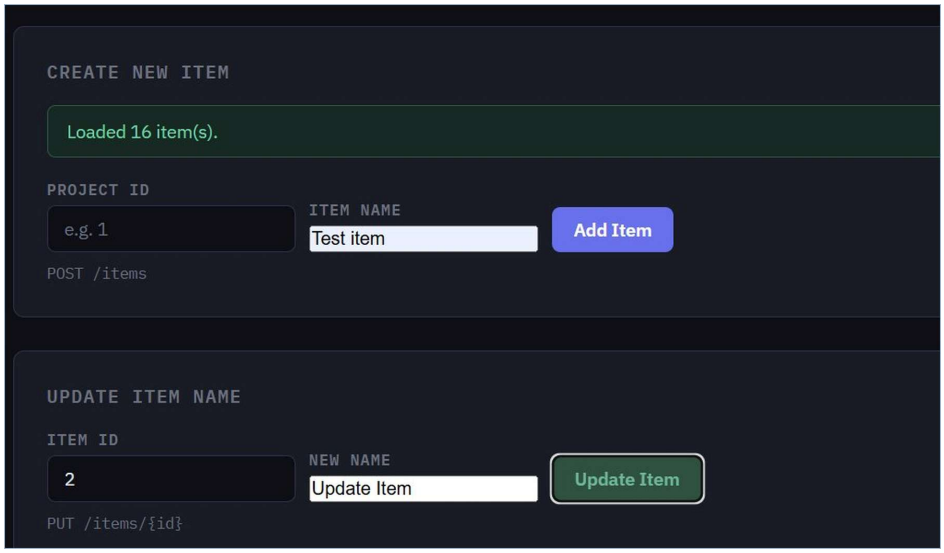


Figure 25 — Frontend: Create New Item (Loaded 16 items after another insert) + Update Item (id=2, name="Update Item") confirming PUT worked

DELETE — DELETE /items/{id}

Delete buttons are present on every row of the Items table. Clicking removes the item immediately.

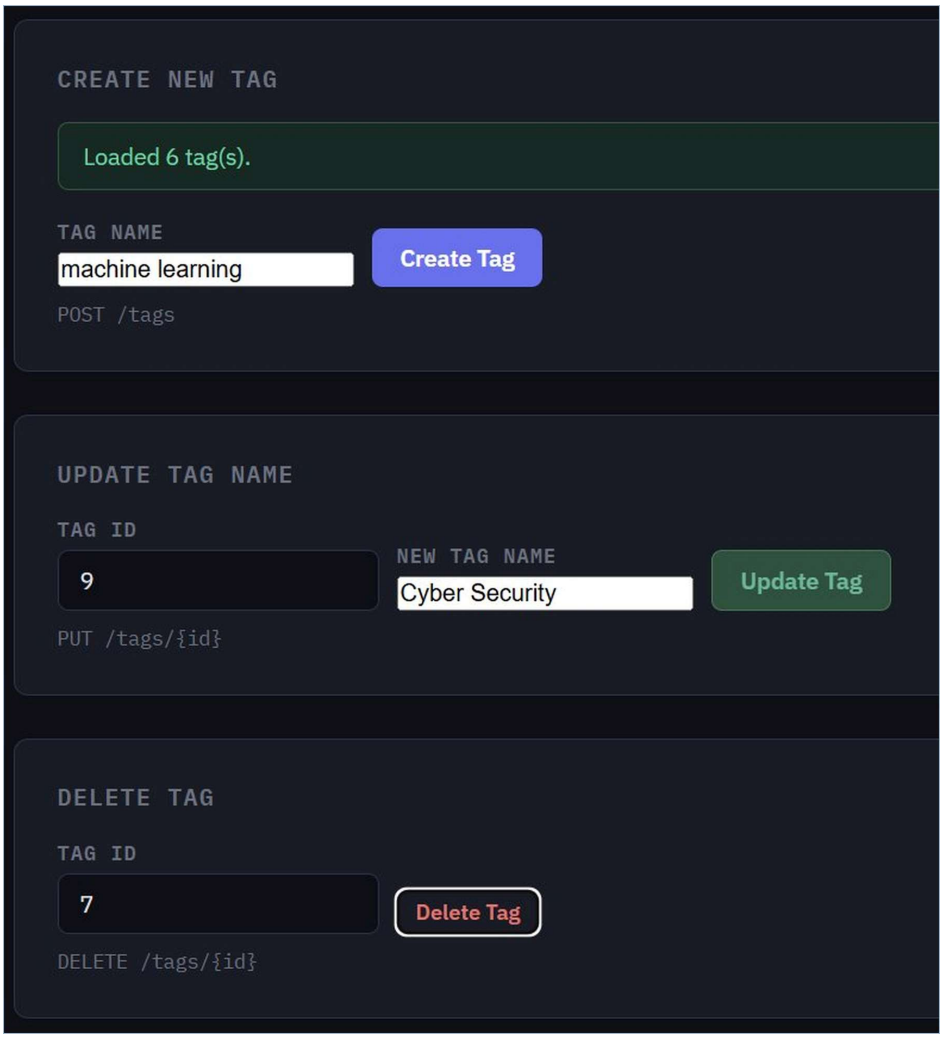


Figure 26 — Frontend: Items tab — Create, Update Item Name, and Delete Item sections all visible and functional

5.4 Tags — Full CRUD Test

CREATE — POST /tags

A new tag "machine learning" was created via the Create New Tag form.

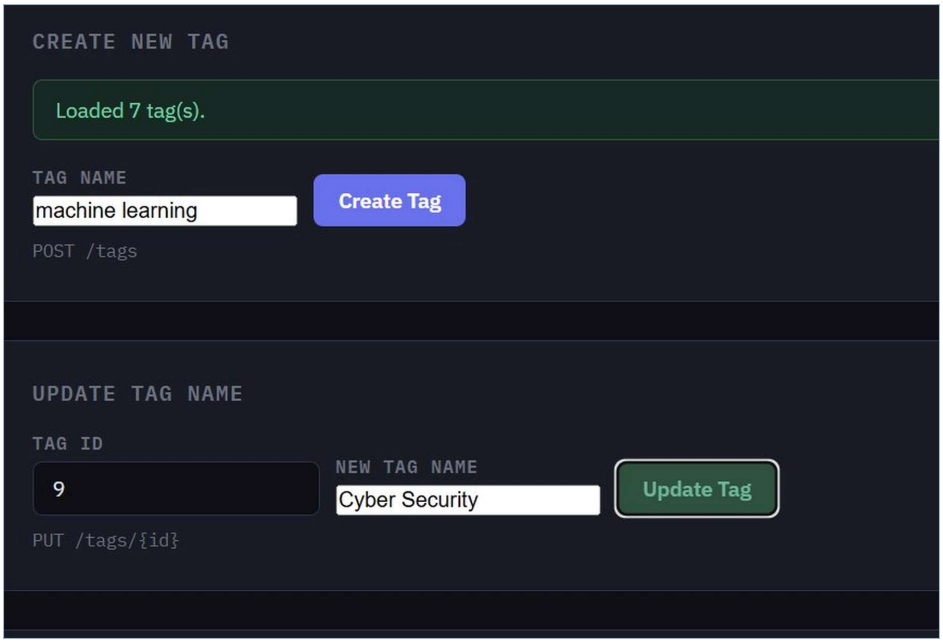
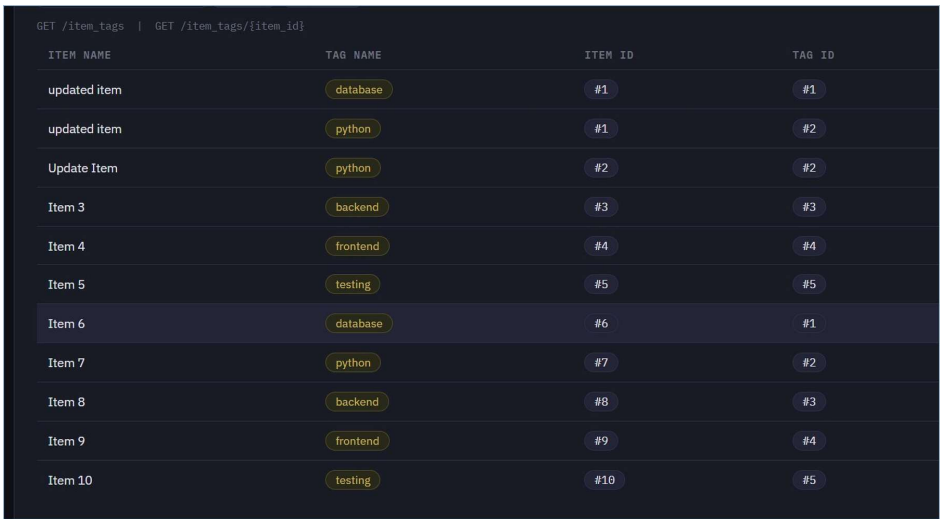


Figure 27 — Frontend: Create New Tag form — tag_name="machine learning", "Loaded 7 tag(s)", and Update Tag Name form (id=9, name="Cyber Security")

READ — GET /tags



GET /item_tags | GET /item_tags/{item_id}

ITEM NAME	TAG NAME	ITEM ID	TAG ID
updated item	database	#1	#1
updated item	python	#1	#2
Update Item	python	#2	#2
Item 3	backend	#3	#3
Item 4	frontend	#4	#4
Item 5	testing	#5	#5
Item 6	database	#6	#1
Item 7	python	#7	#2
Item 8	backend	#8	#3
Item 9	frontend	#9	#4
Item 10	testing	#10	#5

Figure 28 — Frontend: Item Tags table — item_name, tag_name (database/python/backend/frontend/testing), item_id, tag_id all displayed

UPDATE — PUT /tags/{id} and DELETE — DELETE /tags/{id}

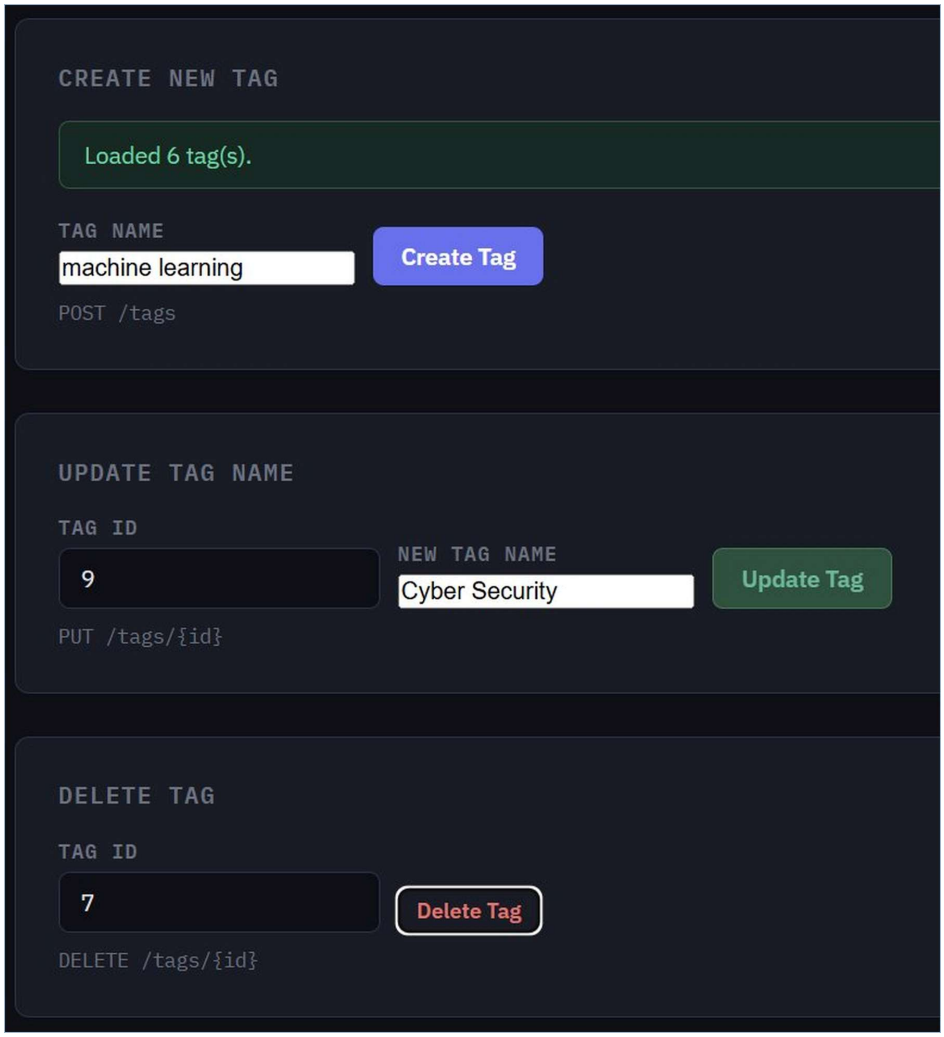


Figure 29 — Frontend: Create New Tag + Update Tag Name (id=9, Cyber Security) + Delete Tag (id=7) — all three operations confirmed in UI

5.5 ItemTags — Link / Unlink Test

CREATE LINK — POST /item_tags

Item #2 was linked to Tag #3 via the Link Tag to Item form.

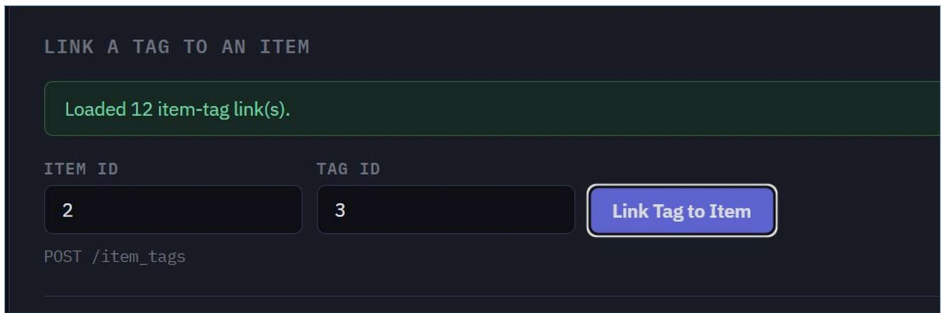


Figure 30 — Frontend: Link Tag to Item form — item_id=2, tag_id=3, "Loaded 12 item-tag link(s)" confirming POST /item_tags

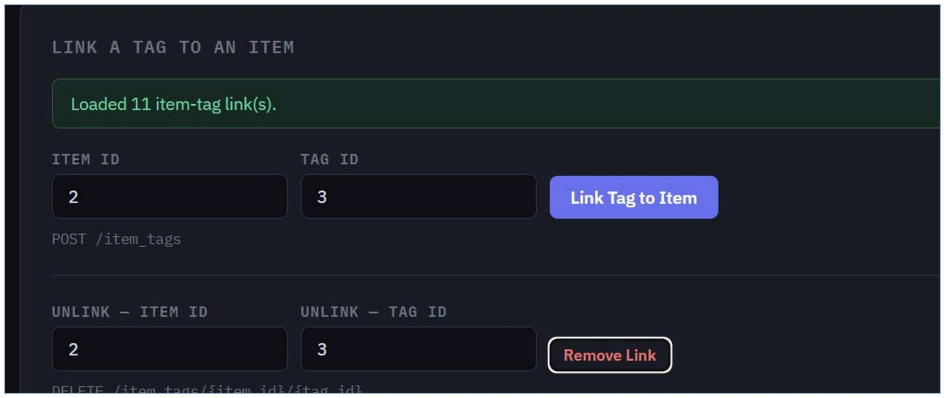


Figure 31 — Frontend: Link form + Unlink (Remove Link) form both visible — item_id=2, tag_id=3

READ — GET /item_tags

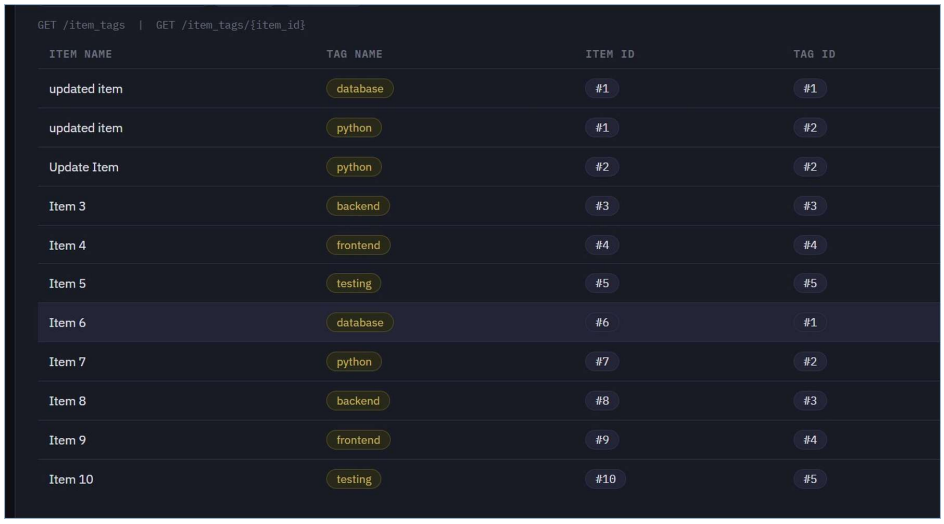


Figure 32 — Frontend: Item Tags table — 10 rows with item_name, tag_name badge, item_id, tag_id confirming GET /item_tags

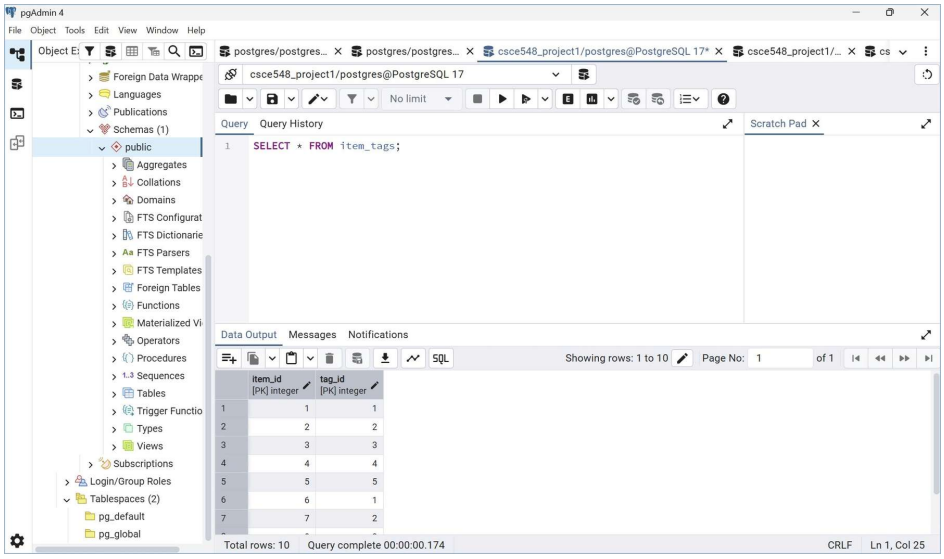


*Figure 33 — pgAdmin: SELECT * FROM item_tags — 10 rows total, query complete in 0.174s confirming database persistence*

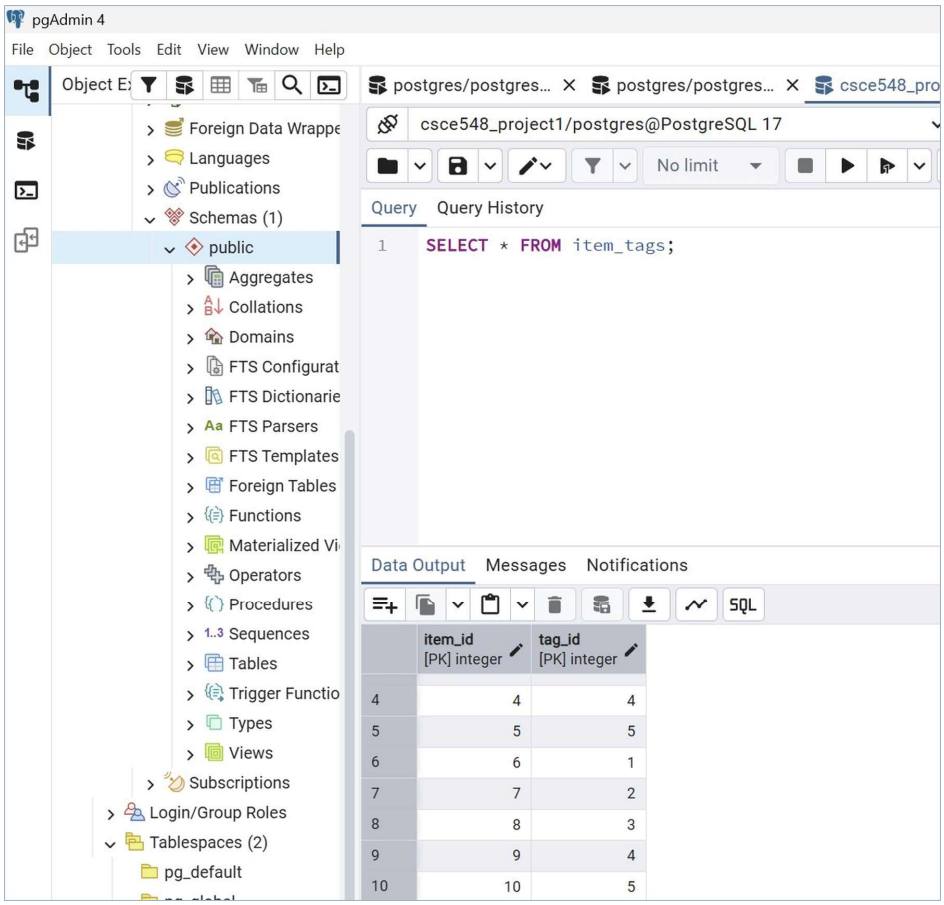


Figure 34 — pgAdmin: item_tags rows 4–10 confirming all item-tag relationships stored correctly

6. Deployment Verification Checklist

Use this checklist to confirm everything is working end-to-end after setup:

Check	How to Verify	Expected Result
Backend running	Open http://127.0.0.1:8080/docs	Swagger UI loads with all 5 endpoint groups
Database connected	Call GET /users in Swagger	Returns JSON array (even if empty)
Frontend loads	Open http://127.0.0.1:8000	5-tab navigation bar visible in browser
CORS working	Click Load All on any tab	Data populates table; no CORS errors in console
POST works	Submit Create form for any entity	Success message; new record appears in table
GET all works	Click Load All on any tab	Full list of records returned
GET by ID works	Enter ID and click Get button	Single matching record returned
PUT works	Submit Update form for any entity	Record updated in table and in pgAdmin
DELETE works	Click Delete button on any row	Row removed from table and pgAdmin

7. Known Issues & Fixes Applied

Issue	Root Cause	Fix Applied
Method Not Allowed on POST /items	Route conflict: /items/{id} declared before /items/project/{id} in service.py	Reordered route declarations — specific paths before generic /{id} patterns
Missing update functions	update_project, update_item, update_tag entirely omitted from business.py and db.py by AI	Added all three update functions manually to both files
Wrong BASE URL in script.js	AI generated frontend pointing to port 8000 instead of backend port 8080	Updated BASE constant to http://127.0.0.1:8080
Field name mismatch on POST /items	Frontend sent { item_name } but backend Pydantic model expected { name }	Fixed createItem() payload to send { project_id, name }

8. Project Structure

```
CSCE-548-PROJECT/
├── src/
│   ├── __init__.py
│   ├── service.py       # FastAPI routes — Service Layer (Tier 3)
│   ├── business.py      # Business logic & validation — Business Layer (Tier 2)
│   └── db.py            # SQL/ORM queries — Data Layer (Tier 1)
├── frontend/
│   ├── index.html       # Single-page UI with 5 tabs
│   └── script.js        # All API calls and DOM logic
├── schema.sql           # PostgreSQL CREATE TABLE statements
├── requirements.txt     # Python dependencies
└── README.md            # GitHub Readme
```

9. Graduate Credit — AI Tool Analysis

This section documents the exact prompts used to generate each layer, every manual change made to the AI output, and a detailed analysis of the AI tool's effectiveness — as required for graduate credit.

9.1 AI Tool Used

Primary tool: ChatGPT (GPT-4o). All prompts were issued per architectural layer. Every generated file was compiled, tested, and corrected before use. No layer was deployed as-is without review.

9.2 Prompts Used to Generate Each Layer

Data Layer — db.py

Prompt: "Generate a Python file called db.py that connects to a PostgreSQL database using psycopg2 and SQLAlchemy. Include complete CRUD functions (get all, get by id, create, update, delete) for

users, projects, items, tags, and item_tags tables. Use a hardcoded DATABASE_URL connection string for local development."

Business Layer — business.py

Prompt: "Generate a Python file called business.py that imports all functions from db.py and wraps them in a business logic layer. Add input validation so required fields cannot be empty. Include full CRUD wrappers for users, projects, items, tags, and item_tags."

Service Layer — service.py

Prompt: "Generate a FastAPI service.py that imports from business.py and exposes REST API endpoints for users, projects, items, tags, and item_tags. Include GET, POST, PUT, and DELETE methods for each entity. Add CORS middleware allowing all origins. Auto-expose Swagger documentation at /docs."

Client Layer — frontend/

Prompt: "Generate an HTML + JS single-page client that calls a FastAPI backend at http://127.0.0.1:8080. Include 5 tabs: Users, Projects, Items, Tags, and Item Tags. Each tab must support Create, Read All, Read by ID, Update, and Delete operations. Apply a dark theme with monospace labels."

9.3 Changes Made to AI-Generated Output

Layer	Change Made	Reason
service.py	Reordered routes: /items/project/{id} moved above /items/{id}	FastAPI matched the wrong route causing Method Not Allowed on POST /items
business.py	Added update_project(), update_item(), update_tag() manually	AI omitted all three update functions from the business layer entirely
db.py	Added SQL update functions for project, item, and tag	Required to back the manual additions in business.py
script.js	Changed BASE URL constant from port 8000 to 8080	AI generated the incorrect backend port
script.js	Fixed createItem() payload: { item_name } → { project_id, name }	Field name mismatch between frontend payload and backend Pydantic schema

9.4 AI Effectiveness Analysis

What AI Did Well

- Generated complete FastAPI structure with Pydantic models, CORS middleware, and Swagger docs in a single pass.
- Produced consistent naming conventions across all layers (user_id, project_id, tag_id, etc.).
- Created a fully functional dark-themed 5-tab frontend requiring only minor styling adjustments.

- Correctly applied REST HTTP method conventions (GET/POST/PUT/DELETE) throughout all layers.
- Generated boilerplate SQL CRUD for all five entities quickly and consistently.

What AI Missed or Got Wrong

- Omitted all three update functions (update_project, update_item, update_tag) from both business.py and db.py — requiring full manual implementation of six functions.
- Generated the wrong backend port (8000 instead of 8080) in the frontend BASE URL constant.
- Created a route declaration ordering conflict in service.py causing silent Method Not Allowed failures.
- Used mismatched field names (item_name vs name) between the frontend payload and backend Pydantic schema.

Conclusion

AI code generation reduced development time for repetitive CRUD scaffolding by an estimated 60-70% but required careful manual review and hands-on testing at every layer before any code could be deployed. Every single generated file contained at least one defect that only surfaced during actual runtime execution. The tool is best used as a first-draft generator — reliable for structure and convention, but not for correctness or completeness without experienced human verification. Graduate-level understanding of REST APIs, FastAPI routing, and relational schema design was essential to identify and fix all AI errors.

10. Limitations & Future Improvements

10.1 Current Limitations

- Local deployment only — no cloud hosting configured in this version.
- No authentication or authorization — all API endpoints are publicly accessible.
- Single HTML file frontend with no build pipeline, bundling, or minification.
- No automated test suite — all testing was performed manually via browser and pgAdmin.

10.2 Future Improvements

- Add JWT-based authentication to secure all API endpoints.
- Deploy the backend to a cloud platform such as Render, Railway, or AWS.
- Containerize the application using Docker and Docker Compose for reproducible deployments.
- Add an automated API test suite using pytest and httpx.
- Migrate the frontend to React or Vue.js for a more maintainable SPA architecture.
- Implement database schema migrations using Alembic.